

北京交通大学考试试题(A卷)

参考答案与评分标准

课程名称: 算法设计与分析 学年学期: 2023—2024 学年第 2 学期

课程编号: M210004B 开课学院: 软件学院 出题教师: 刘铎, 李令昆

第一部分、单项选择。请选择最适合的答案, 并填涂到答题卡上。

(共 24 分)

1.	2.	3.	4.	5.	6.	7.	8.
A	C	D	B	D	B	D	A

第二部分、建模与计算。(共 26 分)

9. (共 8 分) 解: 用 0-1 变量 $s_1 \sim s_9$ 分别表示是否挖石板 $S_1 \sim S_9$, 0-1 变量 $g_1 \sim g_4$ 分别表示是否拿取金板 $G_1 \sim G_4$, 1 表示挖/拿取, 0 表示不挖/不拿取选择。(2 分)

则可得到线性规划模型如下(目标 2 分、等式组 2 分、变量类型约束 2 分)。

$$\text{MAX } 15g_1 + 10g_2 + 20g_3 + 25g_4 - 6s_1 - 5s_2 - 2s_3 - 4s_4 - 3s_5 - 7s_6 - 2s_7 - s_8 - 16s_9$$

$$\text{ST } s_1 + s_2 + s_4 + s_5 - 4g_1 \geq 0$$

$$s_2 + s_3 + s_5 + s_6 - 4g_2 \geq 0$$

$$s_4 + s_5 + s_7 + s_8 - 4g_3 \geq 0$$

$$s_5 + s_6 + s_8 + s_9 - 4g_4 \geq 0$$

$$s_i \in \{0, 1\}, g_j \in \{0, 1\}, \text{ 其中 } 1 \leq i \leq 9, 1 \leq j \leq 4$$

10. (共 10 分) 解: $K(i, w)$ 表示仅考虑金板 $1 \sim i$ 且背包重量限制为 w 时所能取得的最大值, 题目的目标为 $K(4, 15)$ 。 $S(i, w)$ 表示仅考虑金板 $1 \sim i$ 且背包重量限制为 w 时、取得最大值时, 是否选取了物品 i , 1 表示是, 0 表示否。解题过程如表 1 和表 2 所示。允许的最大总价值为 38, 取物品 1、2 和 4。

表 1 $K(i, w)$

$i \backslash w$	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	10	10	10	10	10	10	10	10	10	10	10	10	10	10
2	0	0	10	10	10	10	20	20	20	20	20	20	20	20	20	20
3	0	0	10	10	10	10	20	20	22	22	22	22	32	32	32	32
4	0	0	10	10	10	10	20	20	22	22	22	28	32	32	32	38

表 2 $S(i, w)$

$i \backslash w$	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1
2	0	0	0	0	0	0	1	1	1	1	1	1	1	1	1	1
3	0	0	0	0	0	0	0	0	1	1	1	1	1	1	1	1
4	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	1

11. (共 8 分) 解: 本题目实质上就是 Huffman 编码问题, 过程参见图 1。

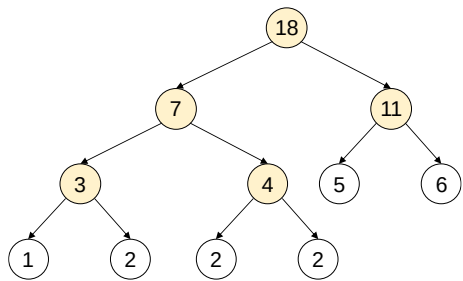


图 1 题 11 解答用图

具体方案为:

- ①将长为 18 的木板锯为长为 7 和 11 的两块, 开销为 18;
 - ②将长为 7 的木板锯为长为 3 和 4 的两块, 开销为 7;
 - ③将长为 3 的木板锯为长为 1 和 2 的两块, 开销为 3;
 - ④将长为 4 的木板锯为长为 2 和 2 的两块, 开销为 4;
 - ⑤将长为 11 的木板锯为长为 5 和 6 的两块, 开销为 11;
- 总 (最小) 开销为 43。

第三部分、算法设计与分析。(共 50 分)

12. (共 12 分) 解: (过程不唯一)

(1) 设 $m(i, j)$ 表示按要求切割从位置 p_i 到位置 p_j 的木板所用的最小开销, 其中 $0 \leq i < j \leq N$ 。

递推关系及初值为:

$$m(i, j) = \begin{cases} 0 & , \text{ if } j - i = 1 \\ \min \{ m(i, k) + m(k, j) + p_j - p_i \} & , i + 1 \leq k \leq j - 1 \end{cases}$$

题目目标为 $m(0, N)$ 。

伪代码为:

输入: L 、 n 、 L_1 、 L_2 、 $\dots$ 、 L_N

输出: 此时的最小总开销

1. for $i=0$ to $N-1$
 2. $m(i, i+1) \leftarrow 0$
 3. for $l=2$ to N
 4. for $i=0$ to $N-l$
 5. $j \leftarrow i+l$
 6. $m(i, j) \leftarrow +\infty$
 7. for $k=i+1$ to $j-1$
 8. if $m(i, k)+m(k, j)+p_j-p_i < m(i, j)$ then
 9. $m(i, j) \leftarrow m(i, k)+m(k, j)+p_j-p_i$
 10. return $m(0, N)$
- (2) $p_0=0$, $p_1=7$, $p_2=9$, $p_3=15$, $p_4=20$, $p_5=30$ 。

表 3 $m(i, j)$

$i \backslash j$	1	2	3	4	5
0	0	9	23	40	68
1	/	0	8	21	44
2	/	/	0	11	32
3	/	/	/	0	15
4	/	/	/	/	0

13. (共 12 分) 解: (过程不唯一)

参考伪代码如下 (8 分):

- IS-VALID($A[1..n]$, $target$) //主函数
1. initialize cnt[1..50] with 0
 2. $maxn \leftarrow \max(A[1], A[2], \dots, A[n])$ //表示最大元素的值
 3. $minn \leftarrow \min(A[1], A[2], \dots, A[n])$ //表示最小元素的值
 4. $total \leftarrow A[1]+A[2]+\dots+A[n]$
 5. for $i=1$ to n do
 6. $cnt[A[i]] \leftarrow cnt[A[i]] + 1$ //表示按元素大小统计个数
 7. return dfs($maxn, n, 0, target, total, maxn, minn$)

dfs (i, num, sum, target, total, maxn, minn)

// num 变量表示还要考虑几个不同的元素大小, sum 变量表示当前考虑的子数组/子序列中元素的和, i 表示下一个备选数字是多少

```
1.  if num = 0 then return true
2.  if sum = target then
3.      return dfs (maxn, n, 0, target, total, maxn, minn)
4.  if sum + minn > len or sum + total < target then
5.      return false
6.  for j = i down to minn do
7.      if cnt[j] > 0 and sum + j ≤ target then
8.          cnt[j] ← cnt[j] - 1
9.          if dfs (j, num-1, sum+j, target, total-j, maxn, minn) then
10.              return true
11.          cnt[j] ← cnt[j] + 1
12.          if sum = 0 or sum + j = target then
13.              break
14. return false
```

算法思路：回溯搜索每个数是否需要选择到某个子数组中。应从最大数到最小数逐个搜索，因为这样可以将剩余问题尽可能的减小。

剪枝依据（4 分，只要回溯过程正确且剪枝依据非平凡均可得分）：

1. 如果当前子数组和加上最小元素的值仍然大于 **target**，不用继续往下搜索了。（dfs 函数的第 4 行）
2. 如果当前子数组的和加上剩余的总长度仍然小于 **target**，也不用继续往下搜索了。（dfs 函数的第 4 行）
3. 如果当前子数组为空，则当前搜索过程中选择的元素必须在该子数组中，否则不存在解。（dfs 函数的第 12 行）
4. 如果当前搜索的数字加上子数组的和正好等于 **target**，但是加上它之后剩余的元素不能分成若干个子数组且和均为 **target**，则不存在解（dfs 函数的第 12 行）。
5. 在搜索的过程中，若当前元素决定不被放入到当前子数组中，则相同的元素也都不再放入到当前子数组中。（IS-VALID 第 6 行按元素大小统计个数，及 dfs 函数的第 6、9 行，选择该元素时下一个状态可以继续选择该元素，而不选择该元素时则直接跳过，搜索进入下一个不同的数字）。

14. (共 14 分) 解: 令 $t_i = c_i - v_i$, 其中 $1 \leq i \leq n$ 。将所有商品分为两类: I类商品满足 $t_i \leq 0$ (推广走量); II类商品满足 $t_i > 0$ 。

(1) (3 分) 施行两阶段贪心策略: 第一阶段针对I类商品, 按照定价低者优先, 依次买完所有商品; 第二阶段针对II类商品, 按照返现高者优先, 依次买完所有商品。

(2) (8 分) 参考伪代码如下:

```

1.  list  $A \leftarrow \Lambda, B \leftarrow \Lambda$  //将序列  $A$  和  $B$  初始化为空序列,
                                     //将分别存放I类商品, 按和II类商品
2.  for  $i = 1$  to  $n$ 
3.       $t_i = c_i - v_i$ ,
4.      if  $t_i \leq 0$  then append  $(c_i, v_i)$  to list  $A$ 
5.      else append  $i$  to list  $B$ 
6.  sort  $A$  according to  $c_i$  non-decreasingly
7.  sort  $B$  according to  $v_i$  non-increasingly
8.  return  $A \circ B$  //返回  $A$ 、 $B$  两个序列的连接

```

(3) (3 分, 以下为详细完整的证明, 学生答出其大意即可。)

假设 $s_1, s_2, \dots, s_n$ 是购买所有商品的一个顺序。定义其中的三类“逆序”:

第①类逆序: 如果存在 $1 \leq x < y \leq n$ 而商品 s_x 属于第II类且商品 s_y 属于第I类, 则称此时 (s_x, s_y) 构成一个第①类逆序。

第②类逆序: 如果存在 $1 \leq x < y \leq n$ 且商品 s_x 和商品 s_y 都属于第I类且 $c_{s_x} > c_{s_y}$, 则称此时 (s_x, s_y) 构成一个第②类逆序。

第③类逆序: 如果 $1 \leq x < y \leq n$ 且商品 s_x 和商品 s_y 都属于第II类且 $v_{s_x} < v_{s_y}$, 则称此时 (s_x, s_y) 构成一个第③类逆序。

假设 S^* 是能够保证购买所有商品且初始钱数最少的所有解中三类逆序数之和最小的解之一, 其购买顺序为 $i_1, i_2, \dots, i_n$ 。

观察结果 1: 如果一个完成任务的顺序中有一个第①类逆序, 那么就一定存在某个 $1 \leq x < n$ 使得 (s_x, s_{x+1}) 构成一个第①类逆序。(学生可直接使用此结论而不加证明。)

结论 1: S^* 中不存在第①类逆序。

采用反证法。假设 S^* 中存在第①类逆序, 则由观察结果 1 可知 S^* 中存在第①类逆序 (s_x, s_{x+1}) , 其中 $1 \leq x < n$ 。商品 s_x 属于第II类且商品 s_{x+1} 属于第I类。

记在购买商品 s_x 之前苗苗手中的钱数为 w , 则有 $w \geq c_{s_x}$; 且如图 2(a)所示, 在购买

商品之后苗苗手中的钱数为 $w - c_{s_x} + v_{s_x}$ 。由 S^* 的定义可知 $w - c_{s_x} + v_{s_x} \geq c_{s_{x+1}}$ 。

交换商品 s_x 和商品 s_{x+1} 的购买次序后,如图2(b)所示。由第II类商品的定义可知 $c_{s_x} > v_{s_x}$,因此由 $w - c_{s_x} + v_{s_x} \geq c_{s_{x+1}}$ 可得 $w \geq c_{s_{x+1}} + (c_{s_x} - v_{s_x}) > c_{s_{x+1}}$,于是用 w 元可以购买商品 s_{x+1} ;购买商品 s_{x+1} 之后,苗苗手中的钱数为 $w - c_{s_{x+1}} + v_{s_{x+1}}$,由第I类商品的定义可知 $c_{s_{x+1}} \leq v_{s_x}$,因此 $w - c_{s_{x+1}} + v_{s_{x+1}} \geq w \geq c_{s_x}$,即仍然可以购买商品 s_x 。

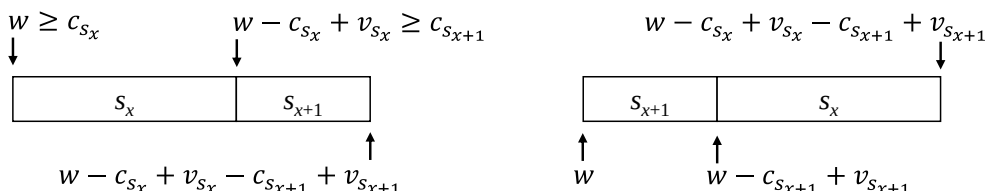


图2 题14解答用图1

因此,交换商品 s_x 和商品 s_{x+1} 的购买次序后,仍然构成保证购买所有商品的解且初始钱数不会增加,但是交换商品 s_x 和商品 s_{x+1} 的购买次序导致总逆序数严格减少,于是和 S^* 的假设产生矛盾。

由此可知能够保证购买所有商品且初始钱数最少的所有解中三类逆序数之和最小的解之一必定是如图3所示的形式。



图3 题14解答用图2

在此基础上,可以得到以下两个观察结果及两个结论。

观察结果 2: 如果一个完成任务的顺序中有一个第②类逆序,那么就一定存在某个 $1 \leq x < n$ 使得 (s_x, s_{x+1}) 构成一个第②类逆序。(学生可直接使用此结论而不加证明。)

结论 2: S^* 中不存在第②类逆序。

采用反证法。假设 S^* 中存在第②类逆序,则由观察结果2可知 S^* 中存在第②类逆序 (s_x, s_{x+1}) ,其中 $1 \leq x < n$ 。

记在购买商品 s_x 之前苗苗手中的钱数为 w ,则有 $w \geq c_{s_x}$;且如图2(a)所示,在购买商品之后苗苗手中的钱数为 $w - c_{s_x} + v_{s_x}$ 。由 S^* 的定义可知 $w - c_{s_x} + v_{s_x} \geq c_{s_{x+1}}$ 。

交换商品 s_x 和商品 s_{x+1} 的购买次序后,如图2(b)所示。由第②类逆序的定义可知 $c_{s_x} \geq c_{s_{x+1}}$,因此用 w 元可以购买商品 s_{x+1} ;购买商品 s_{x+1} 之后,苗苗手中的钱数为 $w - c_{s_{x+1}} + v_{s_{x+1}}$,由 $w - c_{s_x} \geq 0$ 及 $v_{s_{x+1}} - c_{s_{x+1}} \geq 0$ 可知 $(w - c_{s_{x+1}} + v_{s_{x+1}}) - c_{s_x} \geq 0$,即仍然可以购买商品 s_x 。

因此，交换商品 s_x 和商品 s_{x+1} 的购买次序后，仍然构成保证购买所有商品的解且初始钱数不会增加，但是交换商品 s_x 和商品 s_{x+1} 的购买次序导致总逆序数严格减少，于是和 S^* 的假设产生矛盾。

观察结果 3： 如果一个完成任务的顺序中有一个第③类逆序，那么就一定存在某个 $1 \leq x < n$ 使得 (s_x, s_{x+1}) 构成一个第③类逆序。（学生可直接使用此结论而不加证明。）

结论 3： S^* 中不存在第③类逆序。

采用反证法。假设 S^* 中存在第③类逆序，则由观察结果 3 可知 S^* 中存在第③类逆序 (s_x, s_{x+1}) ，其中 $1 \leq x < n$ 。

记在购买商品 s_x 之前苗苗手中的钱数为 w ，则有 $w \geq c_{s_x}$ ；且如图 2(a)所示，在购买商品之后苗苗手中的钱数为 $w - c_{s_x} + v_{s_x}$ 。由 S^* 的定义可知 $w - c_{s_x} + v_{s_x} \geq c_{s_{x+1}}$ 。

交换商品 s_x 和商品 s_{x+1} 的购买次序后，如图 2(b)所示，由第③类逆序的定义可知 $v_{s_x} < v_{s_{x+1}}$ 。由第Ⅱ类商品的定义可知 $c_{s_x} > v_{s_x}$ ，因此由 $w - c_{s_x} + v_{s_x} \geq c_{s_{x+1}}$ 可得 $w \geq c_{s_{x+1}} + (c_{s_x} - v_{s_x}) > c_{s_{x+1}}$ ，于是用 w 元可以购买商品 s_{x+1} ；购买商品 s_{x+1} 之后，苗苗手中的钱数为 $w - c_{s_{x+1}} + v_{s_{x+1}}$ ，可得 $(w - c_{s_{x+1}} + v_{s_{x+1}}) - c_{s_x} > (w - c_{s_x} + v_{s_x}) - c_{s_{x+1}} \geq 0$ ，即仍然可以购买商品 s_x 。

因此，交换商品 s_x 和商品 s_{x+1} 的购买次序后，仍然构成保证购买所有商品的解且初始钱数不会增加，但是交换商品 s_x 和商品 s_{x+1} 的购买次序导致总逆序数严格减少，于是和 S^* 的假设产生矛盾。

结论 4： S^* 中不存在任何一类逆序中的任何一个，于是其就是贪婪解。

15.（共 12 分）**解：**（过程不唯一）本质上就是找到一个位置 l ， $A[l] > A[l+1]$ 。

（1）（6 分）

FIND-CIRCULAR-SHIFT(A, n)

```

1.   $L \leftarrow 1, R \leftarrow n$ 
2.  while  $R - L > 1$  Do
3.       $M \leftarrow (L+R)/2$ 
4.      if  $A[L] < A[M]$  then
5.           $L \leftarrow M$ 
6.      else
7.           $R \leftarrow M$ 
8.  return  $L$ 
```

(2) (3 分) 由于数组中的所有元素均不相等且循环右移前数组单调上升, 于是若数组被循环右移, 则一定有 $A[l] > A[n]$ 。只要找出满足 $A[l] > A[l+1]$ 的 l 即可 (下文称其为“间断点”)。我们采用二分查找方法, 不包含间断点的“一半”一定是单调增的 (于是“左端”的值小于“右端”的值), 而包含间断点的“一半”的值则有如图 4 中所示的特点 (于是“左端”的值大于“右端”的值)。

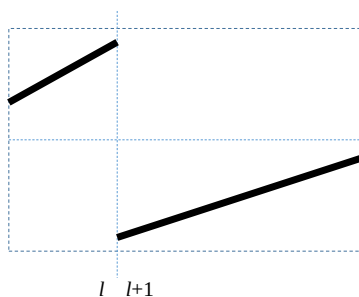


图 4 题 15 解答用图

在算法第 2~7 行的每次迭代中, 都确保间断点的位置 (下标/索引) 在 L (含) ~ R (不含) 中, 直至寻找间断点的范围只包含数组中两项时, 它们必然就是 $A[l]$ 和 $A[l+1]$ 。

(3) (3 分)

用 $T(n)$ 表示输入为 n 时, 递推式为 $T(n) = T\left(\frac{n}{2}\right) + n$ ($n > 2$)、 $T(2) = 0$, 其解为 $T(n) \in \Theta(\log_2 n)$ 。